

Project 1: Container

Cloud Computing (Spring 2026)

Student Name: **Adewole Adeoshun**

Cuny ID: 24081306

Student Type: Undergraduate Student

Professor: Jun Li

Tasks Completed: 1-6

Due Date: March 5, 2026

Platform: Local Environment (Minikube)

Kubernetes Version: v1.35.1

Docker Version: 29.1.4

TASK 1: CONTAINERIZATION

Application Source Code (app.py):

```
from flask import Flask
import os
import socket

app = Flask(__name__)

@app.route("/")
def hello():
    html = "<h3>Hello, Cloud!</h3>" \
        "<b>Hostname:</b> {hostname}<br/>"
    return html.format(hostname=socket.gethostname())

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)
```

Dockerfile:

FROM python:3.11-slim

WORKDIR /app

COPY app.py .

RUN pip install --no-cache-dir flask

EXPOSE 8080

CMD ["python", "app.py"]

Commands Run:

Step 1: Create app.py

```
cat > app.py << 'EOF'
from flask import Flask
import os
import socket

app = Flask(__name__)

@app.route("/")
def hello():
    html = "<h3>Hello, Cloud!</h3>" \
        "<b>Hostname:</b> {hostname}<br/>"
    return html.format(hostname=socket.gethostname())

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)
EOF
```

Step 2: Create Dockerfile

```
cat > Dockerfile << 'EOF'
FROM python:3.11-slim
```

```
WORKDIR /app
```

```
COPY app.py .
```

```
RUN pip install --no-cache-dir flask
```

```
EXPOSE 8080
```

```
CMD ["python", "app.py"]
EOF
```

Step 3: Build Docker image

```
docker build -t flask-app:v1 .
```

Output:

```
[+] Building 7.8s (10/10) FINISHED
docker:desktop-linux
=> [internal] load build definition from Dockerfile
0.0s
=> => transferring dockerfile: 167B
0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim
0.9s
=> [auth] library/python:pull token for registry-1.docker.io
0.0s
=> [internal] load .dockerignore
0.0s
=> => transferring context: 2B
0.0s
=> [1/4] FROM
docker.io/library/python:3.11-slim@sha256:4057d02a202f69bfbfe10f65300519f612eb00fc5
95b8499f77d3cfe5b1b9fd4 2.7s
```

```
=> => resolve
docker.io/library/python:3.11-slim@sha256:4057d02a202f69bfbfe10f65300519f612eb00fc5
95b8499f77d3cfe5b1b9fd4 0.0s
=> => sha256:f378e81ff313c648c10b945bc6e947fdeb51431300952528219df8382d52c4de
250B / 250B 0.1s
=> => sha256:7976ed97cbf4ee7b85a5869ec9adc74a73240241362f8d6e406a2e3ff373b54b
14.31MB / 14.31MB 0.5s
=> => sha256:c45f5486ef43b5210c68f8c7891b71371d42811a9fe71cee67996c5ddfb7721c
1.27MB / 1.27MB 0.4s
=> =>
sha256:3b66ab8c894cad95899b704e688938517870850391d1349c862c2b09214acb86
30.14MB / 30.14MB
```

Step 4: Verify image was created

```
docker images | grep flask-app
```

Output:

```
flask-app:v1 7c0fa7469479 236MB 51.4MB
```

Step 5: Run container locally for testing

```
docker run -d -p 8080:8080 --name flask-test flask-app:v1
```

Step 6: Verify container is running

```
docker ps
```

Output:

```
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES 5f8632f2b7fe
flask-app:v1 "python app.py" 17 seconds ago Up 17 seconds 0.0.0.0:8080->8080/tcp
flask-test
```

Step 7: Test in browser

- Accessed <http://localhost:8080>
- Verified application displays "Hello, Cloud!" and hostname

Step 8: Stop and remove test container

`docker stop flask-test`

`docker rm flask-test`

Output:

`flask-test`

`flask-test`

Screenshots for Task 1:

- Browser showing Flask app running in Docker container at <http://localhost:8080> with "Hello, Cloud!" message and container hostname visible (Task 1 validation)



Explanation:

I chose Python 3.11-slim as the base image because it provides a lightweight Python runtime while keeping the overall image size relatively small (about 236MB). The slim version removes unnecessary system packages, which helps reduce the attack surface and makes builds and deployments faster.

I installed Flask using pip with the `--no-cache-dir` flag so that pip doesn't store its package cache inside the image. This helps keep the final Docker image smaller and cleaner.

The application runs on 0.0.0.0:8080. Using 0.0.0.0 instead of localhost (127.0.0.1) is important because it allows the Flask server to accept connections from outside the container. If it were bound to localhost, Docker's port mapping and Kubernetes networking wouldn't be able to reach the application.

I also exposed port 8080 in the Dockerfile to clearly indicate which port the application is expected to run on. Before deploying the container to Kubernetes, I first ran and tested the container locally with Docker to make sure everything worked correctly. Validating the container locally helped confirm that the Dockerfile and application setup were correct before adding the additional complexity of Kubernetes orchestration.

TASK 2: CLUSTER PROVISIONING

Commands Run:

Step 1: Install Minikube
brew install minikube

Step 2: Verify Minikube installation

minikube version

Output:

minikube version: v1.38.1

commit: c93a4cb9311efc66b90d33ea03f75f2c4120e9b0

Step 3: Start Minikube cluster

minikube start

Output:

😊 minikube v1.38.1 on Darwin 26.3 (arm64)
✨ Automatically selected the docker driver
🔑 Using Docker Desktop driver with root privileges
👍 Starting "minikube" primary control-plane node in "minikube" cluster
🚚 Pulling base image v0.0.50 ...
📦 Downloading Kubernetes v1.35.1 preload ...
🔥 Creating docker container (CPUs=2, Memory=3072MB) ...
🐳 Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
🔗 Configuring bridge CNI (Container Networking Interface) ...
🔍 Verifying Kubernetes components...
☀️ Enabled addons: default-storageclass, storage-provisioner
🏁 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default

Step 4: Verify cluster is running

kubectl get nodes

Output:

NAME	STATUS	ROLES	AGE	VERSION
Minikube	Ready	control-plane	3m2s	v1.35.1

Step 5: Get cluster information

kubectl cluster-info

Output:

Kubernetes control plane is running at https://127.0.0.1:61082

CoreDNS is running at

https://127.0.0.1:61082/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

Screenshots for Task 2:

- Terminal showing **kubectl get nodes** output with minikube node in Ready state (Task 2 validation)

```
dewole@Dewoles-MacBook-Pro Project 1 % kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
minikube    Ready     control-plane  25m   v1.35.1
dewole@Dewoles-MacBook-Pro Project 1 % kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:61082
CoreDNS is running at https://127.0.0.1:61082/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
dewole@Dewoles-MacBook-Pro Project 1 %
```

Explanation:

I used Minikube for local Kubernetes development so I could learn container orchestration without dealing with cloud costs or extra setup complexity. Minikube lets me run a single-node Kubernetes cluster locally, which still gives me a full Kubernetes environment but runs directly on my machine.

In my setup, the cluster runs Kubernetes v1.35.1 with 2 CPUs and 3GB of memory, which is enough to run multiple pod replicas for testing and development. Minikube also automatically configured kubectl, the Kubernetes command-line tool, so it could communicate with the local cluster. I verified that everything was working by running `kubectl get nodes`, which showed the node in Ready status.

The control plane running at 127.0.0.1:61082 confirms that the Kubernetes API server is accessible locally. Using this setup makes it easy for me to experiment, deploy containers, and test configurations quickly before potentially deploying the application to a cloud-based Kubernetes environment in production.

TASK 3: APPLICATION DEPLOYMENT

Commands Run:

Step 1: Load Docker image into Minikube

minikube image load flask-app:v1

Step 2: Verify image is available in Minikube

minikube image ls | grep flask-app

Output:

docker.io/library/flask-app:v1

Step 3: Create deployment.yaml

```
cat > deployment.yaml << 'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-app
  labels:
    app: flask-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: flask-app
  template:
    metadata:
      labels:
        app: flask-app
    spec:
      containers:
      - name: flask-app
        image: flask-app:v1
        imagePullPolicy: Never
        ports:
```

```
- containerPort: 8080
EOF
```

Deployment Configuration (deployment.yaml):

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-app
  labels:
    app: flask-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: flask-app
  template:
    metadata:
      labels:
        app: flask-app
    spec:
      containers:
        - name: flask-app
          image: flask-app:v1
          imagePullPolicy: Never
          ports:
            - containerPort: 8080
```

Step 4: Apply deployment

```
kubectl apply -f deployment.yaml
```

Output:
deployment.apps/flask-app created

Step 5: Verify deployment

```
kubectl get deployments
```

Output:

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
flask-app	2/2	2	2	14s

Step 6: Check pods status

kubectl get pods

Output:

NAME	READY	STATUS	RESTARTS	AGE
flask-app-78dc567fd7-8fzhz	1/1	Running	0	20s
flask-app-78dc567fd7-v448m	1/1	Running	0	20s

Screenshots for Task 3:

- Terminal showing **kubectl get pods** with both Flask app pods in Running status (Task 3 validation)

```
dewole@Dewoles-MacBook-Pro Project 1 % kubectl get deployments
NAME          READY    UP-TO-DATE    AVAILABLE    AGE
flask-app     2/2      2             2            14s
dewole@Dewoles-MacBook-Pro Project 1 % kubectl get pods
NAME          READY    STATUS    RESTARTS    AGE
flask-app-78dc567fd7-8fzhz  1/1      Running   0           20s
flask-app-78dc567fd7-v448m  1/1      Running   0           20s
dewole@Dewoles-MacBook-Pro Project 1 %
```

Explanation:

I configured the deployment with 2 replicas so the application has basic high availability. If one pod fails for any reason, the other can continue handling requests, which keeps the service running.

Since I'm using Minikube locally, an important configuration is setting `imagePullPolicy: Never`. This tells Kubernetes to use the Docker image that already exists locally instead of trying to pull it from a remote registry like Docker Hub. Because of this, I first loaded the image into Minikube using `minikube image load`. Without that step, the pods would fail with `ImagePullBackOff` errors since Minikube cannot access images that are only built on my local Docker daemon.

The selector with `matchLabels: app: flask-app` connects the deployment to the pods it manages, allowing Kubernetes to track and manage the correct set of pods for that deployment.

I also defined `containerPort: 8080`, which tells Kubernetes which port the container is listening on. This becomes important later when configuring services to route traffic to the pods.

After applying the configuration, both pods successfully reached `Running` status with no errors, confirming that the deployment was configured correctly and that the application is ready to handle incoming traffic.

TASK 4: SERVICE EXPOSURE

Commands Run:

Step 1: Create service.yaml

```
cat > service.yaml << 'EOF'
apiVersion: v1
kind: Service
metadata:
  name: flask-app-service
spec:
  type: NodePort
  selector:
    app: flask-app
  ports:
```

```
- protocol: TCP
  port: 8080
  targetPort: 8080
  nodePort: 30080
EOF
```

Service Configuration (service.yaml):

```
apiVersion: v1
kind: Service
metadata:
  name: flask-app-service
spec:
  type: NodePort
  selector:
    app: flask-app
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 8080
      nodePort: 30080
```

Step 2: Apply service

```
kubectl apply -f service.yaml
```

Output:

```
service/flask-app-service created
```

Step 3: Verify service creation

```
kubectl get services
```

Output:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
Flask-app-service	NodePort	10.102.47.248	<none>	8080:30080/TCP	6s
Kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	39m

Step 4: Get accessible URL

minikube **service** flask-app-service --url

Output:

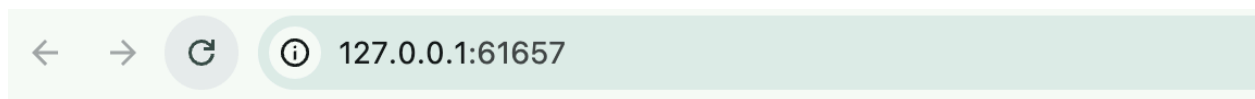
http://127.0.0.1:61657

Step 5: Access application in browser

- Opened http://127.0.0.1:61657
- Refreshed multiple times to observe different hostnames
- Confirmed load balancing across pods

Screenshots for Task 4:

- Browser showing Flask app at http://127.0.0.1:61657 with first pod hostname

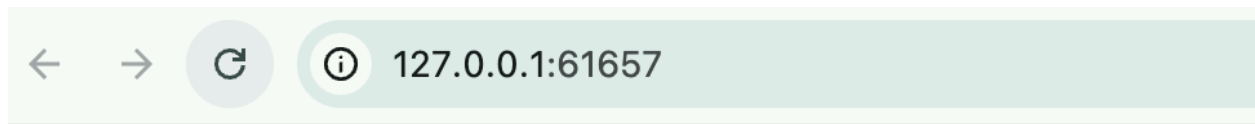


Hello, Cloud

Reload this page

Hostname: flask-app-78dc567fd7-8fzhz

- Browser showing Flask app with DIFFERENT pod hostname after refresh



Hello, Cloud!

Hostname: flask-app-78dc567fd7-v448m

Explanation:

I chose NodePort as the service type because it's the simplest way to expose an application when running Kubernetes locally with Minikube. NodePort opens a specific port on the node (30080) and forwards incoming traffic to the Kubernetes service.

The service uses the selector app: flask-app to determine which pods should receive traffic. This matches the labels defined in the deployment, allowing Kubernetes to correctly route requests to the pods that belong to this application.

The port configuration includes three parts. port: 8080 represents the internal service port inside the Kubernetes cluster. targetPort: 8080 is the port on the container where the Flask application is actually listening. Finally, nodePort: 30080 exposes the service externally on the node so it can be accessed from outside the cluster.

Because Minikube runs Kubernetes inside Docker, I used the command `minikube service flask-app-service --url` to generate a reachable URL. This command automatically sets up the necessary port forwarding so the service can be accessed from my host machine.

To confirm that load balancing was working, I refreshed the browser multiple times and saw different pod hostnames in the response. This showed that Kubernetes was distributing traffic across both replicas using round-robin load balancing, confirming that the service is correctly routing external traffic to the internal pods.

TASK 5: LIFECYCLE MANAGEMENT (SCALING & UPDATING)

Commands Run:

Part A: Scaling

Step 1: Scale deployment from 2 to 4 replicas

```
kubectl scale deployment flask-app --replicas 4
```

Output:

```
deployment.apps/flask-app scaled
```

Step 2: Verify scaling

```
kubectl get pods
```

Output:

NAME	READY	STATUS	RESTARTS	AGE
Flask-app-78dc567fd7-4qrfn	1/1	Running	0	7s
flask-app-78dc567fd7-8fzhz	1/1	Running	0	10m
Flask-app-78dc567fd7-v448m	1/1	Running	0	10m
Flask-app-78dc567fd7-zxmjk	1/1	Running	0	7s

Step 3: Check deployment status

kubectl get deployments

Output:

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
flask-app	4/4	4	4	10m

Screenshots for Task 5:

- Terminal showing **kubectl get pods** with all 4 pods in Running status (Task 5 - Scaling validation)

```
dewole@dewoles-MacBook-Pro Project 1 % kubectl scale deployment flask-app --replicas=4
deployment.apps/flask-app scaled
dewole@dewoles-MacBook-Pro Project 1 % kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
flask-app-78dc567fd7-4qrfn          1/1     Running   0           7s
flask-app-78dc567fd7-8fzhz          1/1     Running   0          10m
flask-app-78dc567fd7-v448m          1/1     Running   0          10m
flask-app-78dc567fd7-zxmjk          1/1     Running   0           7s
dewole@dewoles-MacBook-Pro Project 1 %
```

Part B: Rolling Update

Step 1: Modify app.py with new message

```
cat > app.py << 'EOF'
from flask import Flask
import os
import socket

app = Flask(__name__)

@app.route("/")
def hello():
    html = "<h3>Hello, Spring 2026!</h3>" \
        "<b>Hostname:</b> {hostname}<br/>"
    return html.format(hostname=socket.gethostname())
```

```
if __name__ == "__main__":  
    app.run(host="0.0.0.0", port=8080)  
EOF
```

Step 2: Build version 2 image

```
docker build -t flask-app:v2 .
```

Output:

```
[+] Building 6.0s (10/10) FINISHED  
=> [internal] load build definition from Dockerfile  
=> [internal] load metadata for docker.io/library/python:3.11-slim  
=> [1/4] FROM docker.io/library/python:3.11-slim  
=> CACHED [2/4] WORKDIR /app  
=> [3/4] COPY app.py .  
=> [4/4] RUN pip install --no-cache-dir flask  
=> exporting to image  
=> naming to docker.io/library/flask-app:v2
```

Step 3: Load v2 image into Minikube

```
minikube image load flask-app:v2
```

Step 4: Verify both versions in Minikube

```
minikube image ls | grep flask-app
```

Output:

```
docker.io/library/flask-app:v2  
docker.io/library/flask-app:v1
```

Step 5: Perform rolling update

```
kubectl set image deployment/flask-app flask-app=flask-app:v2
```

Output:

deployment.apps/flask-app image updated

Step 6: Monitor rollout status

kubectl rollout status deployment/flask-app

Output:

deployment "flask-app" successfully rolled out

Step 7: Verify new pods are running

```
dewole@dewoles-MacBook-Pro Project 1 % kubectl set image deployment/flask-app flask-app=flask-app:v2
deployment.apps/flask-app image updated
dewole@dewoles-MacBook-Pro Project 1 % kubectl rollout status deployment/flask-app
deployment "flask-app" successfully rolled out
dewole@dewoles-MacBook-Pro Project 1 % kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
flask-app-78dc567fd7-4qrfn         1/1     Terminating    0          7m17s
flask-app-78dc567fd7-8fzhz         1/1     Terminating    0          17m
flask-app-78dc567fd7-v448m         1/1     Terminating    0          17m
flask-app-78dc567fd7-zxmjk         1/1     Terminating    0          7m17s
flask-app-d4bcfcf4d-4kcv4          1/1     Running         0          14s
flask-app-d4bcfcf4d-dglk7          1/1     Running         0          13s
flask-app-d4bcfcf4d-kx7vr          1/1     Running         0          14s
flask-app-d4bcfcf4d-zbvt7          1/1     Running         0          12s
dewole@dewoles-MacBook-Pro Project 1 % kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
flask-app-d4bcfcf4d-4kcv4          1/1     Running         0          91s
flask-app-d4bcfcf4d-dglk7          1/1     Running         0          90s
flask-app-d4bcfcf4d-kx7vr          1/1     Running         0          91s
flask-app-d4bcfcf4d-zbvt7          1/1     Running         0          89s
dewole@dewoles-MacBook-Pro Project 1 %
```

Step 8: Get updated service URL

minikube service flask-app-service --url

Output:

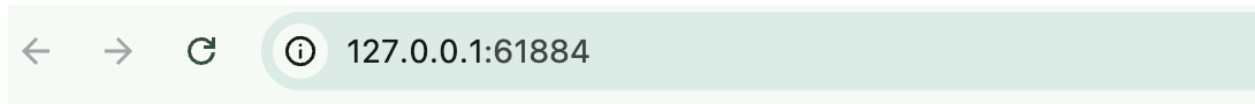
http://127.0.0.1:61884

Step 9: Verify updated application in browser

- Accessed http://127.0.0.1:61884
- Confirmed "Hello, Spring 2026!" message displayed
- Verified new pod hostnames (d4bcfcf4d-*)

Screenshots for Task 5:

- Browser showing updated message "Hello, Spring 2026!" with new pod hostname (Task 5 - Rolling Update validation)



Hello, Spring 2026!

Hostname: flask-app-d4bcfcf4d-dglk7

Explanation:

I demonstrated Kubernetes' elasticity by scaling the deployment from 2 to 4 replicas using a single command. Once I updated the replica count, Kubernetes automatically created two additional pods to match the desired state. The scaling happened within seconds and did not require any downtime or manual intervention. In a real production environment, this type of horizontal scaling allows applications to handle increased traffic by adding more pod instances, instead of vertically scaling by increasing resources on a single instance. Because Kubernetes uses a declarative model, I only needed to specify the desired number of replicas and Kubernetes handled the rest, including scheduling, creating, and managing the new pods.

Rolling Update:

I also performed a zero-downtime rolling update by modifying the application message to "Hello, Spring 2026!" and building a new v2 container image. I then used the `kubectl set image` command to update the deployment. This triggered Kubernetes' default rolling update strategy, where old pods are gradually replaced with new ones. During the rollout, I observed both the old pods (78dc567fd7-) and the new pods (d4bcfcf4d-) running at the same time. This shows how Kubernetes replaces pods incrementally while ensuring that enough healthy replicas remain available to handle traffic.

At no point did the deployment drop below the minimum required replicas, which guarantees zero downtime during updates. This approach is much safer than a recreate strategy, where all old pods are terminated before new ones start, potentially causing service interruption. After the rollout completed, all four pods were running the updated v2 image, which I verified by checking the pod names and seeing the updated message in the browser. This demonstrates how Kubernetes can safely deploy application updates in production without affecting availability.

TASK 6: TEARDOWN

Commands Run:

Step 1: Delete service

```
kubectl delete service flask-app-service
```

Output:

```
service "flask-app-service" deleted from default namespace
```

Step 2: Delete deployment

```
kubectl delete deployment flask-app
```

Output:

```
deployment.apps "flask-app" deleted from default namespace
```

Step 3: Verify resources are deleted

```
kubectl get all
```

Output:

NAME	READY	STATUS	RESTARTS	AGE
pod/flask-app-d4bcfcf4d-4kcv4	1/1	Terminating	0	5h17m
pod/flask-app-d4bcfcf4d-dglk7	1/1	Terminating	0	5h17m
pod/flask-app-d4bcfcf4d-kx7vr	1/1	Terminating	0	5h17m
pod/flask-app-d4bcfcf4d-zbvt7	1/1	Terminating	0	5h17m

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	6h8m

Step 4: Stop Minikube cluster

minikube stop

Output:

 Stopping node "minikube" ...
 Powering off "minikube" via SSH ...
 1 node stopped.

Explanation:

I performed a complete teardown of the environment by first deleting the Kubernetes resources and then stopping the Minikube cluster. I started by deleting the service, which removed the network endpoint responsible for routing external traffic to the pods. After that, I deleted the deployment, which caused Kubernetes to terminate the running pods.

When the deployment was deleted, all four pods entered the “Terminating” state, allowing the application to shut down gracefully and finish any in-flight requests before stopping. Once those resources were removed, the only remaining service in the cluster was the default kubernetes service, which is the internal API service used by the control plane.

Finally, I stopped the Minikube cluster using `minikube stop`, which shuts down the virtual node and frees the CPU and memory that were allocated to the cluster. This step is important during local development because it prevents the cluster from consuming system resources when it’s not being used.

In a cloud environment, performing this type of teardown would be even more important because unused resources could continue running and generate unnecessary costs. By

removing all deployments, services, and stopping the cluster, I ensured the environment was fully cleaned up and returned to a fresh state with no leftover resources.

SUMMARY

This project successfully demonstrated the complete lifecycle of containerizing and orchestrating a cloud-native application using Docker and Kubernetes. Key accomplishments include:

- **Containerization:** Created a minimal, production-ready Docker image using Python 3.11-slim base
- **Local Kubernetes:** Provisioned a Minikube cluster for development without cloud costs
- **High Availability:** Deployed application with multiple replicas across pods
- **Load Balancing:** Verified traffic distribution using Kubernetes Service
- **Elasticity:** Demonstrated horizontal scaling from 2 to 4 replicas
- **Zero-Downtime Updates:** Performed rolling update to new application version
- **Proper Cleanup:** Removed all resources to free system resources

Platform Choice Justification: I chose Minikube (local environment) over cloud platforms (EKS/GKE) because it provides a complete Kubernetes environment without incurring cloud costs, allows rapid iteration during development, and teaches the same Kubernetes concepts that apply to production cloud deployments. The `imagePullPolicy: Never` configuration and `minikube image load` commands were specific to local development, but all other Kubernetes configurations (deployment, service, scaling, rolling updates) are identical to what would be used in production cloud environments.